GURU99 BANK AUTOMATION FRAMEWORK

Project Documentation

GURU99
Demo Site

Field	Details
Project Title	Guru99 Bank Automation Framework
Author	Shahid Raza
Organization	Wipro
Technology Stack	Java, Selenium, TestNG, Maven, Docker, Jenkins, Extent Reports
Date	June 2026

Table of Contents

1. Project Overview

1.1 Objective

The objective of this project is to automate the testing of the Guru99 Banking application using Selenium WebDriver and TestNG, following the Page Object Model (POM) design pattern.

1.2 Features Automated

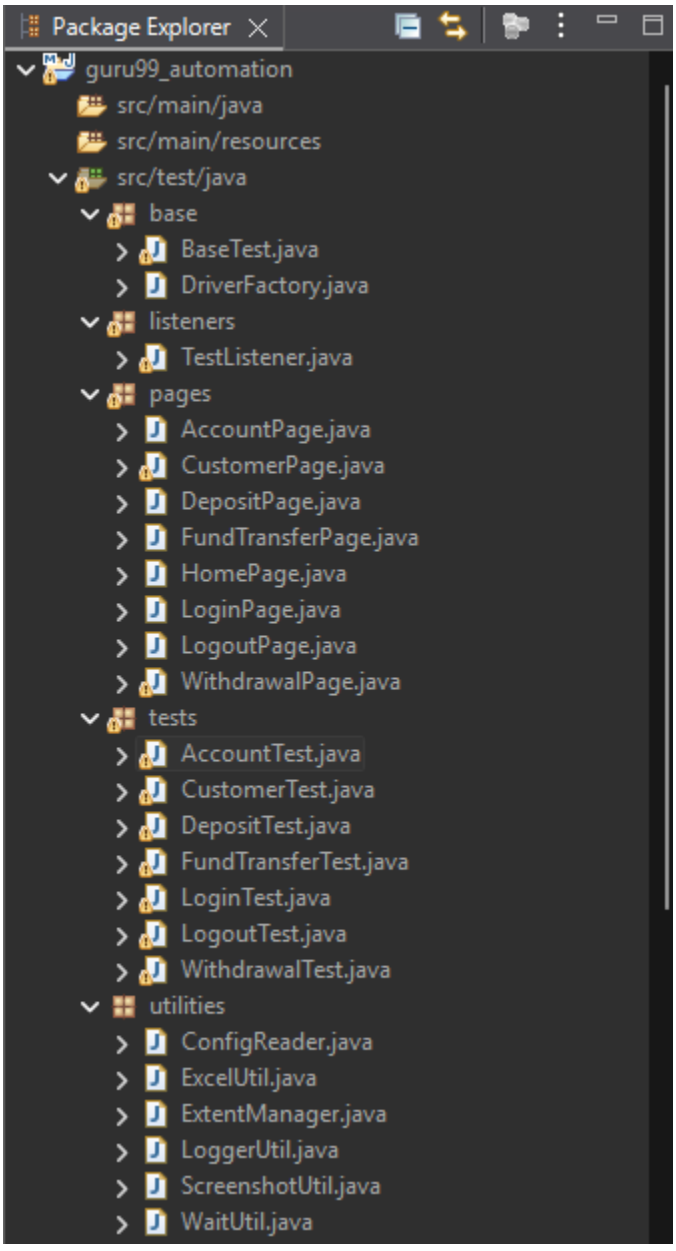
- Login
- Customer Management
  - Add Customer
- Account Management
  - Create Account
- Deposit
- Withdrawal
- Fund Transfer
- Logout

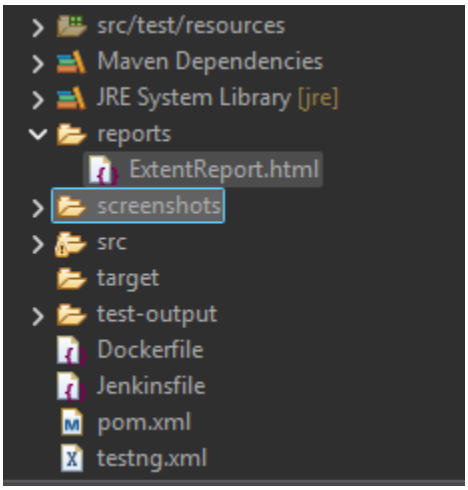
2. Technology Stack

Technology	Purpose
Java	Programming Language
Selenium WebDriver	UI Automation
TestNG	Test Execution
Maven	Build Management
Apache POI	Excel Handling
Extent Reports	Reporting
Docker	Containerization
Jenkins	CI/CD
Git/GitHub	Version Control

3. Project Structure

The diagram below illustrates the overall directory structure of the automation framework.





3.1 src/main/java

base

BaseTest.java

Purpose:

- Initializes browser
- Performs login
- Handles teardown

Code:

```
package base;

import org.openqa.selenium.WebDriver;
import org.testng.annotations.AfterClass;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Optional;
import org.testng.annotations.Parameters;

import pages.LoginPage;

public class BaseTest {

    protected WebDriver driver;

    protected LoginPage loginPage;

    @Parameters("browser")
    @BeforeClass
    public void setup(@Optional("chrome")String browser) {

        driver = DriverFactory.initDriver(browser);

        driver.get("https://demo.guru99.com/V4/");
```

```
        loginPage = new LoginPage(driver);

    }

    protected void loginToApplication() {

        loginPage.enterUsername("mngr662995");

        loginPage.enterPassword("Uzutynu");

        loginPage.clickLogin();
    }


    @AfterClass
    public void tearDown() {
       System.out.println("test completed");
//         DriverFactory.quitDriver();
    }



}
```

DriverFactory.java

Purpose:

- Creates and manages WebDriver instance
- Supports browser initialization

```
package base;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;
import org.openqa.selenium.edge.EdgeDriver;
import org.openqa.selenium.edge.EdgeOptions;
import org.openqa.selenium.firefox.FirefoxDriver;
import org.openqa.selenium.firefox.FirefoxOptions;

public class DriverFactory {

    private static WebDriver driver;

    public static WebDriver initDriver(String browser) {

        if (driver == null) {

            switch (browser.toLowerCase()) {

                case "chrome":
```

```
                    ChromeOptions chromeOptions = new ChromeOptions();

                    chromeOptions.addArguments("--headless=new");
                    chromeOptions.addArguments("--no-sandbox");
                    chromeOptions.addArguments("--disable-dev-shm-usage");
                    chromeOptions.addArguments("--disable-gpu");
                    chromeOptions.addArguments("--window-size=1920,1080");

                    System.out.println("Creating Chrome Driver");

                    driver = new ChromeDriver(chromeOptions);
//                  driver = new ChromeDriver();

                    break;

                case "edge":

                    EdgeOptions edgeOptions = new EdgeOptions();

                    edgeOptions.addArguments("--headless=new");
                    edgeOptions.addArguments("--no-sandbox");
                    edgeOptions.addArguments("--disable-dev-shm-usage");
                    edgeOptions.addArguments("--disable-gpu");
                    edgeOptions.addArguments("--window-size=1920,1080");

                    System.out.println("Creating Edge Driver");

                    driver = new EdgeDriver(edgeOptions);
//                  driver = new EdgeDriver();

                    break;

                case "firefox":

                    FirefoxOptions firefoxOptions = new FirefoxOptions();

                    firefoxOptions.addArguments("--headless");

                    System.out.println("Creating Firefox Driver");

                    driver = new FirefoxDriver(firefoxOptions);
//                  driver = new FirefoxDriver();

                    break;

                default:

                    throw new IllegalArgumentException(
                            "Invalid browser: " + browser);
            }

            driver.manage().window().maximize();
        }
```

```
        return driver;
    }

    public static WebDriver getDriver() {

        return driver;
    }

    public static void quitDriver() {

        if (driver != null) {

            driver.quit();

            driver = null;
        }
    }
}
```

listeners

TestListener.java

Purpose:

- Implements TestNG Listeners
- Captures screenshots on failure
- Logs test events

```
package listeners;

import org.testng.ITestContext;
import org.testng.ITestListener;
import org.testng.ITestResult;

import com.aventstack.extentreports.*;
import base.BaseTest;
import base.DriverFactory;
import utilities.ExtentManager;
import utilities.ScreenshotUtil;

public class TestListener implements ITestListener {

    private static ExtentReports extent =
            ExtentManager.getInstance();

    private static ThreadLocal<ExtentTest> test =
            new ThreadLocal<>();

    @Override
    public void onStart(ITestContext context) {

        System.out.println("Execution Started");
    }
```

```
@Override
public void onTestStart(ITestResult result) {
   System.out.println("Listener: Test Started -> "
            + result.getMethod().getMethodName());

    ExtentTest extentTest =
            extent.createTest(
                    result.getMethod()
                            .getMethodName());

    test.set(extentTest);
}

@Override
public void onTestSuccess(ITestResult result) {

   System.out.println("Listener: Test Passed -> "
            + result.getMethod().getMethodName());
    test.get().pass("Test Passed");
}

@Override
public void onTestFailure(ITestResult result) {

    System.out.println("Listener: Test Failed -> "
              + result.getMethod().getMethodName());
    test.get().fail(result.getThrowable());

    try {

        Object currentClass =
                result.getInstance();

        BaseTest base =
                (BaseTest) currentClass;

        String screenshotPath =
                ScreenshotUtil.captureScreenshot(
                            DriverFactory.getDriver(),
                        result.getMethod()
                                .getMethodName());

        test.get().addScreenCaptureFromPath(
                screenshotPath);

    } catch (Exception e) {

        e.printStackTrace();
    }
}

@Override
public void onTestSkipped(ITestResult result) {
```

```
        test.get().skip("Test Skipped");
    }

    @Override
    public void onFinish(ITestContext context) {

        extent.flush();

        System.out.println("Execution Completed");
    }
}
```

pages

LoginPage.java

Purpose:

Contains locators and methods for login functionality.

Methods:

- enterUsername()
- enterPassword()
- clickLogin()

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import utilities.WaitUtil;

public class LoginPage {

    WebDriver driver;
    WaitUtil waitUtil;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        this.waitUtil = new WaitUtil(driver);
    }

    private By username = By.name("uid");
    private By password = By.name("password");
    private By loginbtn = By.name("btnLogin");
    private By welcomeMessage = By.xpath("//marquee[@class='heading3']");

    public void enterUsername(String user1) {
        waitUtil.waitForElementVisible(username).sendKeys(user1);
    }

    public void enterPassword(String pass) {
        waitUtil.waitForElementVisible(password).sendKeys(pass);
    }
```

```
    public void clickLogin() {
        waitUtil.waitForElementClickable(loginbtn).click();
    }

    public void loginToApplication(String user, String pass) {
        enterUsername(user);
        enterPassword(pass);
        clickLogin();
    }

    public boolean isLoginSuccessful() {
        return waitUtil.waitForElementVisible(welcomeMessage)
                .getText()
                .equals("Welcome To Manager's Page of Guru99 Bank");
    }
}
```

CustomerPage.java

Purpose:

Handles customer-related operations.

Methods:

- navigateToNewCustomer()
- createNewCustomer()
- getCustomerId()
- editCustomer()

```
package pages;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.testng.Assert;


import utilities.WaitUtil;

public class CustomerPage {

WebDriver driver;

      public CustomerPage(WebDriver driver) {
            this.driver=driver;
      }


      By newCustBtn=By.linkText("New Customer");

      By custName=By.name("name");
```

```
	By  date=By.id("dob");

	By address= By.name("addr");

	By city=By.name("city");

	By state=By.name("state");

	By pinCode=By.name("pinno");

	By mobNum=By.name("telephoneno");

	By email= By.name("emailid");

	By password=By.name("password");

	By subBtn=By.name("sub");


	 By customerId = By.xpath("//td[text()='Customer ID']/following-sibling::td");




	 public void navigateToNewCustomer() {


//		WaitUtil waitUtil = new WaitUtil(driver);
//
//		WebElement button =
//			waitUtil.waitForElementClickable(newCustBtn);


		WaitUtil waitUtil = new WaitUtil(driver);

		WebElement button =
			waitUtil.waitForElementClickableFluent(newCustBtn);


		JavascriptExecutor js = (JavascriptExecutor) driver;
		js.executeScript("arguments[0].scrollIntoView(true);", button);
		js.executeScript("arguments[0].click();", button);
	}

	public void enterCustomerName(String customerName) {
		driver.findElement(custName).sendKeys(customerName);
	}


	public void enterDOB(String DOB) {
		driver.findElement(date).sendKeys(DOB);
	}
```

```
	public void enterAddress(String add) {
		driver.findElement(address).sendKeys(add);
	}
	public void enterCity(String c) {
		driver.findElement(city).sendKeys(c);

	}

	public void enterState(String st) {
		driver.findElement(state).sendKeys(st);
	}

	public void enterPin(String pin) {
		driver.findElement(pinCode).sendKeys(pin);
	}

	public void enterPhone(String phone) {
		driver.findElement(mobNum).sendKeys(phone);
	}

	public void enterEmail(String e) {
		driver.findElement(email).sendKeys(e);
	}

	public void enterPass(String pass1) {
		driver.findElement(password).sendKeys(pass1);
	}

	public void submitCustomer() {
		driver.findElement(subBtn).click();
	}

	public void createNewCustomer(
			String name,
			String dob,
			String addr,
			String cityName,
			String stateName,
			String pin,
			String phone,
			String emailId,
			String pass) {

		enterCustomerName(name);
		enterDOB(dob);
		enterAddress(addr);
		enterCity(cityName);
		enterState(stateName);
		enterPin(pin);
		enterPhone(phone);
		enterEmail(emailId);
		enterPass(pass);

		submitCustomer();
```

```
    }

    public String getCustomerId() {

        WaitUtil waitUtil = new WaitUtil(driver);

        return waitUtil
                .waitForElementVisibleFluent(customerId)
                .getText();
    }

}
```

AccountPage.java

Purpose:

Handles account creation and validation.

```
package pages;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.testng.Assert;

import utilities.WaitUtil;

public class AccountPage {

    WebDriver driver;

    public AccountPage(WebDriver driver) {
        this.driver = driver;
    }

    // New Account Menu
    By newAccountLink =
            By.linkText("New Account");

    // Form Fields
    By customerId =
            By.name("cusid");

    By accountType =
            By.name("selaccount");

    By initialDeposit =
            By.name("inideposit");

    By submitBtn =
            By.name("button2");
```

```
// Success Page
By accountIdText =
        By.xpath("//td[text()='Account ID']/following-sibling::td");




    public void clickNewAccount() {

              WaitUtil waitUtil = new WaitUtil(driver);
              //
              WebElement button =
                      waitUtil.waitForElementClickable(newAccountLink);


              JavascriptExecutor js = (JavascriptExecutor) driver;
              js.executeScript("arguments[0].scrollIntoView(true);", button);
              js.executeScript("arguments[0].click();", button);
         }

public void clickSubmitBtn() {
   driver.findElement(submitBtn)
    .click();
}

public void createAccount(
        String custId,
        String accType,
        String depositAmount) {

    driver.findElement(customerId)
            .sendKeys(custId);

    driver.findElement(accountType)
            .sendKeys(accType);

    driver.findElement(initialDeposit)
            .sendKeys(depositAmount);

    driver.findElement(submitBtn)
            .click();
}


   public void acceptAccountFailedCreationAlert() {
          Alert alert = WaitUtil.waitForAlert(driver);

           String alertText = alert.getText();

           System.out.println("Alert Message: " + alertText);

           Assert.assertTrue( alertText.contains("Please fill all fields"),
                   "Unexpected alert message");
```

```
                alert.accept();
        }

    public String getAccountId() {

        return driver.findElement(accountIdText)
                .getText();
    }
}
```

DepositPage.java

Purpose:

Automates deposit functionality.

```
package pages;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.testng.Assert;

import utilities.WaitUtil;

public class DepositPage {

    WebDriver driver;

    public DepositPage(WebDriver driver) {
        this.driver = driver;
    }

    By depositLink = By.linkText("Deposit");

    By accountNo = By.name("accountno");

    By amount = By.name("ammount");

    By description = By.name("desc");

    By submitBtn = By.name("AccSubmit");

    By successMessage =
            By.xpath("//p[contains(text(),'Transaction details')]");
```

```
    public void clickDeposit() {

           WaitUtil waitUtil = new WaitUtil(driver);

           WebElement button =
                   waitUtil.waitForElementClickable(depositLink);

        ((JavascriptExecutor) driver)
                .executeScript(
                        "arguments[0].scrollIntoView(true);",
                        button);

        ((JavascriptExecutor) driver)
                .executeScript(
                        "arguments[0].click();",
                        button);
    }

    public void clickDepositSubmitBtn() {
         driver.findElement(submitBtn)
           .click();
    }

    public void depositAmount(
            String accNo,
            String depositAmount,
            String remarks) {

        driver.findElement(accountNo)
                .sendKeys(accNo);

        driver.findElement(amount)
                .sendKeys(depositAmount);

        driver.findElement(description)
                .sendKeys(remarks);

        driver.findElement(submitBtn)
                .click();
    }


    public void acceptDepositeAlert() {
              Alert alert = WaitUtil.waitForAlert(driver);

              String alertText = alert.getText();

              System.out.println("Alert Message: " + alertText);

              Assert.assertTrue( alertText.contains("Please fill all fields"),
                      "Unexpected alert message");

              alert.accept();
       }
```

```
    public boolean isDepositSuccessful() {

        return driver.findElement(successMessage)
                .isDisplayed();
    }
}
```

WithdrawalPage.java

Purpose:

Automates withdrawal functionality.

```
package pages;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.testng.Assert;
import org.testng.asserts.SoftAssert;

import utilities.WaitUtil;

public class WithdrawalPage {

    WebDriver driver;

    public WithdrawalPage(WebDriver driver) {
        this.driver = driver;
    }

    By withdrawalLink =
            By.linkText("Withdrawal");

    By accountNo =
            By.name("accountno");

    By amount =
            By.name("ammount");

    By description =
            By.name("desc");

    By submitBtn =
            By.name("AccSubmit");

    By successMessage =
            By.xpath("//p[contains(text(),'Transaction details')]");



    public void clickWithdrawal() {
```

```
          WaitUtil waitUtil = new WaitUtil(driver);

          WebElement button =
                  waitUtil.waitForElementClickable(withdrawalLink);

        ((JavascriptExecutor) driver)
                .executeScript(
                        "arguments[0].scrollIntoView(true);",
                        button);

        ((JavascriptExecutor) driver)
                .executeScript(
                        "arguments[0].click();",
                        button);
    }


    public void withdrawAmount(
            String accNo,
            String withdrawAmount,
            String remarks) {

        driver.findElement(accountNo)
                .sendKeys(accNo);

        driver.findElement(amount)
                .sendKeys(withdrawAmount);

        driver.findElement(description)
                .sendKeys(remarks);

        driver.findElement(submitBtn)
                .click();
    }

    public boolean isWithdrawalSuccessful() {

        WaitUtil waitUtil = new WaitUtil(driver);
          return waitUtil.waitForElementVisible(successMessage).isDisplayed();
    }

    public void acceptAlert(String msg, SoftAssert softAssert) {

        Alert alert = WaitUtil.waitForAlert(driver);

        String alertText = alert.getText();

        System.out.println("Alert Message: " + alertText);

        softAssert.assertTrue(
                alertText.contains(msg),
                "Unexpected alert message. Expected: [" + msg + "] but got: [" +
alertText + "]");

        alert.accept();
```

```
    }

}
```

FundTransferPage.java

Purpose:

Automates fund transfer operations.

```
package pages;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.testng.Assert;

import utilities.WaitUtil;

public class FundTransferPage {

    WebDriver driver;

    public FundTransferPage(WebDriver driver) {
        this.driver = driver;
    }

    By fundTransferLink =
            By.linkText("Fund Transfer");

    By payerAccount =
            By.name("payersaccount");

    By payeeAccount =
            By.name("payeeaccount");

    By amount =
            By.name("ammount");

    By description =
            By.name("desc");

    By submitBtn =
            By.name("AccSubmit");

    By successMessage =
            By.xpath("//p[contains(text(),'Fund Transfer Details')]");

    public void clickTrasferSubmitBtn() {
       driver.findElement(submitBtn).click();
```

```
    }



//    public void clickFundTransfer() {
//
//        ((JavascriptExecutor) driver)
//                .executeScript("window.scrollTo(0,300)");
//
//        WaitUtil waitUtil = new WaitUtil(driver);
//
//        WebElement button =
//                waitUtil.waitForElementClickable(fundTransferLink);
//
//        button.click();
//    }

    public void clickFundTransfer() {

        WebElement button =
                driver.findElement(fundTransferLink);

        ((JavascriptExecutor) driver)
                .executeScript(
                        "arguments[0].scrollIntoView(true);",
                        button);

        ((JavascriptExecutor) driver)
                .executeScript(
                        "arguments[0].click();",
                        button);
    }

    public void transferFunds(
            String payerAcc,
            String payeeAcc,
            String transferAmount,
            String remarks) {

        driver.findElement(payerAccount)
                .sendKeys(payerAcc);

        driver.findElement(payeeAccount)
                .sendKeys(payeeAcc);

        driver.findElement(amount)
                .sendKeys(transferAmount);

        driver.findElement(description)
                .sendKeys(remarks);

        driver.findElement(submitBtn)
                .click();
    }
```

```
    public boolean isTransferSuccessful() {

        return driver.findElement(successMessage)
                .isDisplayed();
    }


      public void acceptAccNotExist() {
             Alert alert = WaitUtil.waitForAlert(driver);

              String alertText = alert.getText();

              System.out.println("Alert Message: " + alertText);

              Assert.assertTrue( alertText.contains("does not exist!!!"),
                      "Unexpected alert message");

              alert.accept();
      }



      public void acceptMissField() {
             Alert alert = WaitUtil.waitForAlert(driver);

              String alertText = alert.getText();

              System.out.println("Alert Message: " + alertText);

              Assert.assertTrue( alertText.contains("Please fill all fields"),
                      "Unexpected alert message");

              alert.accept();
      }



      public void acceptLowBal() {
             Alert alert = WaitUtil.waitForAlert(driver);

              String alertText = alert.getText();

              System.out.println("Alert Message: " + alertText);

              Assert.assertTrue( alertText.contains("Transfer Failed. Account
Balance low!!"),
                      "Unexpected alert message");

              alert.accept();
      }


      public void acceptAccountNumSame() {
             Alert alert = WaitUtil.waitForAlert(driver);
```

```
            String alertText = alert.getText();

            System.out.println("Alert Message: " + alertText);

            Assert.assertTrue( alertText.contains("Payers account No and Payees
account No Must Not be Same!!!"),
                    "Unexpected alert message");

            alert.accept();
      }



}
```

HomePage.java

Purpose:

Contains home page navigation methods.

```
package pages;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

import utilities.WaitUtil;

public class HomePage {

    WebDriver driver;

    By managerText =
            By.xpath("//td[contains(text(),'Manger Id')]");

    public HomePage(WebDriver driver) {

        this.driver = driver;
    }

    public boolean isManagerDisplayed() {
        WaitUtil waitUtil = new WaitUtil(driver);
           return waitUtil.waitForElementVisible(managerText).isDisplayed();


    }
}
```

LogoutPage.java

Purpose:

Automates logout functionality.

```java
package pages;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

import utilities.WaitUtil;

public class LogoutPage {

    WebDriver driver;

    public LogoutPage(WebDriver driver) {
        this.driver = driver;
    }

    By logoutLink = By.linkText("Log out");



    public void clickLogout() {

        WebElement button =
                driver.findElement(logoutLink);

        ((JavascriptExecutor) driver)
                .executeScript(
                        "arguments[0].scrollIntoView(true);",
                        button);

        ((JavascriptExecutor) driver)
                .executeScript(
                        "arguments[0].click();",
                        button);
    }

    public String getLogoutAlertText() {

        Alert alert = WaitUtil.waitForAlert(driver);

        String alertText = alert.getText();

        alert.accept();

        return alertText;
    }

    public boolean isLoginPageDisplayed() {

        return driver.findElement(By.name("uid"))
                .isDisplayed();
    }
}
```

3.2 tests

LoginTest.java

Description:

Validates login functionality.

```
package tests;

import org.openqa.selenium.Alert;
import org.testng.Assert;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.HomePage;
import pages.LoginPage;
import utilities.ExcelUtil;
import utilities.WaitUtil;


public class LoginTest extends BaseTest {


       LoginPage loginPage;
         @BeforeClass
           public void accountSetup() {

              loginPage=new LoginPage(driver);
//            driver.navigate().to("https://demo.guru99.com/V4/");
           }



    @DataProvider(name = "loginData")
    public Object[][] getLoginData() {

        ExcelUtil excel = new ExcelUtil(
                "src/test/resources/testdata/Guru99Data.xlsx",
                "LoginData");

        return excel.getSheetData();
    }

    @Test(dataProvider = "loginData")
    public void loginTest(String username,
                          String password,
                          String status) {
       driver.navigate().to("https://demo.guru99.com/V4/");

       loginPage.loginToApplication(username, password);
```

```
        if(status.equalsIgnoreCase("valid")) {

            HomePage homePage = new HomePage(driver);

            Assert.assertTrue(
                    homePage.isManagerDisplayed(),
                    "Valid Login Failed");

            System.out.println("Valid Login Passed");

        } else {


              try {

                    if(password.isEmpty()) {
                          loginPage.clickLogin();
                    }



                    Alert alert = WaitUtil.waitForAlert(driver);

                    String alertText = alert.getText();

                    System.out.println("Alert Message: " + alertText);

                    Assert.assertTrue(
                            alertText.contains("User or Password is not valid"),
                            "Unexpected alert message");

                    alert.accept();

                } catch(Exception e) {

                    Assert.fail("Expected alert not displayed");
                }
        }
    }
}
```

CustomerTest.java

Description:

Performs data-driven testing for customer creation using Excel.

```
package tests;

import org.openqa.selenium.Alert;
import org.testng.Assert;
import org.testng.annotations.BeforeClass;
```

```java
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.CustomerPage;
import utilities.ExcelUtil;
import utilities.WaitUtil;


public class CustomerTest extends BaseTest  {

    private CustomerPage customerPage;
    int rowNumber=1;
        @BeforeClass
          public void initPages() {

              customerPage = new CustomerPage(driver);
              loginToApplication();
          }
      @DataProvider(name="customerData")
      public Object[][] getCustomerData(){

          ExcelUtil excel =
              new ExcelUtil(
              "src/test/resources/testdata/CustomerData_Sample.xlsx",
              "CustomerData");

          return excel.getSheetData();
      }


      @Test(dataProvider="customerData")

      public void addCustomerTest(
              String tcId,
              String name,
              String dob,
              String address,
              String city,
              String state,
              String pin,
              String phone,
              String email,
              String password,
              String customerId,
              String executionStatus,
              String expectedresult
              ) {


          customerPage.navigateToNewCustomer();

          String uniqueEmail =
                  System.currentTimeMillis()
                  + email;
```

```
            customerPage.createNewCustomer(
                    name,
                    dob,
                    address,
                    city,
                    state,
                    pin,
                    phone,
                    uniqueEmail,
                    password);

            if(expectedresult.equalsIgnoreCase("valid")) {

                String generatedCustomerId =
                          customerPage.getCustomerId();
                   Assert.assertFalse(generatedCustomerId.isEmpty());

                   ExcelUtil excel =
                           new ExcelUtil(
                           "src/test/resources/testdata/CustomerId.xlsx",
                           "CustomerID");


                   excel.setCellData(
                           rowNumber,
                           0,
                           generatedCustomerId);

                   rowNumber++;

            }else {
                try {

                    Alert alert = WaitUtil.waitForAlert(driver);

                    String alertText = alert.getText();

                    System.out.println("Alert Message: " + alertText);

                    Assert.assertTrue( alertText.contains("please fill all fields"),
                            "Unexpected alert message");

                    alert.accept();

                } catch(Exception e) {

                    Assert.fail("Expected alert not displayed");
                }
            }

        }


}
```

AccountTest.java

Description:

Validates account creation.

```
package tests;

import org.testng.Assert;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.AccountPage;
import utilities.ExcelUtil;


public class AccountTest extends BaseTest {

    AccountPage accountPage;
    int rowNum=1;

    @BeforeClass
    public void accountSetup() {

        accountPage = new AccountPage(driver);
        loginToApplication();
    }

    @DataProvider(name = "accountData")
    public Object[][] getAccountData() {

        ExcelUtil excel =
                new ExcelUtil(
                        "src/test/resources/testdata/NewAccount.xlsx",
                        "accountData");
        return excel.getSheetData();
    }

    @Test(dataProvider = "accountData")

    public void createAccountTest(

            String customerId,
            String accountType,
            String deposit,
            String status) {

        accountPage.clickNewAccount();

        accountPage.createAccount(
```

```
                customerId,
                accountType,
                deposit);

        if(status.equalsIgnoreCase("valid")) {
               String generatedAccountId =
                      accountPage.getAccountId();

              Assert.assertFalse( generatedAccountId.isEmpty(),"Account ID not
generated");

              ExcelUtil excel =
                      new ExcelUtil(
                              "src/test/resources/testdata/AccountId.xlsx",
                              "AccountID");



              excel.setCellData( rowNum, 0,generatedAccountId);
              rowNum++;

              System.out.println(
                      "Generated Account ID : "
                              + generatedAccountId);
        }else {
             if(deposit=="") {
                   accountPage.clickSubmitBtn();
             }
             accountPage.acceptAccountFailedCreationAlert();
        }


    }
}
```

DepositTest.java

Description:

Verifies deposit functionality and defect handling.

```
package tests;

import org.testng.Assert;
import org.testng.annotations.BeforeClass;

import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
import org.testng.asserts.SoftAssert;

import base.BaseTest;

import pages.DepositPage;
import utilities.ExcelUtil;
```

```
public class DepositTest extends BaseTest {

    DepositPage depositPage;




    @BeforeClass
    public void accountSetup() {
       depositPage = new DepositPage(driver);
       loginToApplication();
    }

    @DataProvider(name = "depositData")
    public Object[][] getDepositData() {

        ExcelUtil excel =
                new ExcelUtil(
                        "src/test/resources/testdata/DepositData.xlsx",
                        "DepositData");

        return excel.getSheetData();
    }

@Test(dataProvider = "depositData")

public void depositTest(
        String accountId,
        String amount,
        String description,
        String status) {

    SoftAssert softAssert = new SoftAssert();

    depositPage.clickDeposit();

    depositPage.depositAmount(
            accountId,
            amount,
            description);



    if(status.equalsIgnoreCase("valid")) {

        try {

            boolean success =
                    depositPage.isDepositSuccessful();

            softAssert.assertTrue(
                    success,
```

```
                    "Deposit success page not displayed");

        } catch (Exception e) {

            String pageSource = driver.getPageSource();

//            softAssert.assertFalse(
//                    pageSource.contains("HTTP ERROR 500"),
//                    "Application defect: HTTP ERROR 500 displayed after deposit");
//            if(pageSource.contains("HTTP ERROR 500")) {
//                 driver.navigate().back();
//            }

            boolean isHttp500 = pageSource.contains("HTTP ERROR 500");

            if (isHttp500) {
                System.out.println("[KNOWN DEFECT] HTTP ERROR 500 displayed after
deposit - Bug logged");
                driver.navigate().back();

            } else {
                softAssert.assertFalse(false, "Unexpected error after deposit");
            }

            System.out.println("Deposit functionality appears broken. HTTP 500 page
displayed.");
        }

    }else {
       depositPage.acceptDepositeAlert();
    }
    softAssert.assertAll();
}
}
```

WithdrawalTest.java

Description:

Verifies withdrawal functionality.

```
package tests;

import org.testng.Assert;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
import org.testng.asserts.SoftAssert;

import base.BaseTest;
import pages.DepositPage;
import pages.WithdrawalPage;
import utilities.ExcelUtil;
```

```
public class WithdrawalTest extends BaseTest {

    WithdrawalPage withdrawalPage;


    @BeforeClass
    public void accountSetup() {



       withdrawalPage = new WithdrawalPage(driver);
       loginToApplication();
    }


    @DataProvider(name = "withdrawalData")
    public Object[][] getWithdrawalData() {

        ExcelUtil excel =
                new ExcelUtil(
                        "src/test/resources/testdata/WithdrawalData.xlsx",
                        "WithdrawalData");

        return excel.getSheetData();
    }

    @Test(dataProvider = "withdrawalData")

    public void withdrawalTest(
            String accountId,
            String amount,
            String description,
            String status) {


        withdrawalPage.clickWithdrawal();
        SoftAssert softAssert = new SoftAssert();

        try {

            withdrawalPage.withdrawAmount(
                    accountId,
                    amount,
                    description);

            if(status.equalsIgnoreCase("valid")) {

                Assert.assertTrue(
                        withdrawalPage.isWithdrawalSuccessful());

            } else {
             if(amount.isEmpty()) {

                 withdrawalPage.acceptAlert("Please fill all fields", softAssert);
```

```
            } else if(accountId.startsWith("122545")) {

                  withdrawalPage.acceptAlert("Account does not exist",softAssert);

            } else {

                double withdrawalAmt = Double.parseDouble(amount);

                if(withdrawalAmt > 100000) {

                    withdrawalPage.acceptAlert(
                            "Transaction Failed. Account Balance Low!!!",
                            softAssert);

                } else if(withdrawalAmt < 0) {

                    if(withdrawalPage.isWithdrawalSuccessful()) {
//                                  Assert.fail(
//                                      "DEFECT: Application accepted withdrawal with
negative amount: "
//                                      + withdrawalAmt);

                                System.out.println("[KNOWN DEFECT] Application accepted
withdrawal with negative amount: "
                                        + withdrawalAmt + " - Bug logged.");
                        }

//                          try {
//                               withdrawalPage.acceptAlert("Account does not
exist",softAssert);
//                            } catch (Exception e) {
//                                System.out.println("No alert present");
//                            }
//
//                            Assert.fail("Defect: Application accepted withdrawal
with negative amount");
                }
             }

            }

        } catch(Exception e) {

                Assert.fail(e.getMessage());

        }
        softAssert.assertAll();
    }
}
```

FundTransferTest.java

Description:

Validates fund transfer scenarios.

```
package tests;

import org.openqa.selenium.Alert;
import org.testng.Assert;
import org.testng.annotations.BeforeClass;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.AccountPage;
import pages.FundTransferPage;
import utilities.ExcelUtil;
import utilities.WaitUtil;


public class FundTransferTest extends BaseTest {

    FundTransferPage fundTransferPage;

    @BeforeClass
    public void accountSetup() {

         fundTransferPage =
                   new FundTransferPage(driver);
         loginToApplication();
    }



    @DataProvider(name = "fundTransferData")
    public Object[][] getFundTransferData() {

        ExcelUtil excel =
                new ExcelUtil(
                        "src/test/resources/testdata/FundTrafer.xlsx",
                        "fundTransferData");

        return excel.getSheetData();
    }

    @Test(dataProvider = "fundTransferData")

    public void fundTransferTest(

            String payerAcc,
            String payeeAcc,
            String transferAmount,
            String description,
            String status) {
```

```
        fundTransferPage.clickFundTransfer();

        try {

            fundTransferPage.transferFunds(
                    payerAcc,
                    payeeAcc,
                    transferAmount,
                    description);

            if (status.equalsIgnoreCase("valid")) {


//                Alert alert = WaitUtil.waitForAlert(driver);
//
//                if(alert != null) {
//                   String alertText = alert.getText();
//                     System.out.println("Alert Message: " + alertText);
//                     Assert.assertTrue( alertText.contains("Transfer Failed.
Account Balance low!!"),
//                             "Unexpected alert message");
//
//                   alert.accept();
//                }else {
                  Assert.assertTrue(
                          fundTransferPage.isTransferSuccessful());
//                }

            } else {

              try {

                    if (payerAcc.equalsIgnoreCase(payeeAcc)
                            && !payerAcc.isEmpty()) {
                        fundTransferPage.acceptAccountNumSame();

                    } else if (payerAcc.isEmpty()
                            || payeeAcc.isEmpty()
                            || transferAmount.isEmpty()
                            || description.isEmpty()) {
                         if(description.isEmpty()) {
                               fundTransferPage.clickTrasferSubmitBtn();
                         }
                        fundTransferPage.acceptMissField();

                    } else if (!transferAmount.isEmpty()) {
                        double amount = Double.parseDouble(transferAmount);
                        if (amount > 100000) {
                            fundTransferPage.acceptLowBal();
                        }
                    }


             }catch(Exception e) {
```

```
                Assert.fail("Expected alert not displayed");
            }
              }

        } catch (Exception e) {

             Assert.fail("Expected alert not displayed");
        }
    }

}
```

LogoutTest.java

Description:

Tests logout functionality.

```
package pages;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.JavascriptExecutor;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;

import utilities.WaitUtil;

public class LogoutPage {

    WebDriver driver;

    public LogoutPage(WebDriver driver) {
        this.driver = driver;
    }

    By logoutLink = By.linkText("Log out");




    public void clickLogout() {

        WebElement button =
                driver.findElement(logoutLink);

        ((JavascriptExecutor) driver)
                .executeScript(
                        "arguments[0].scrollIntoView(true);",
                        button);

        ((JavascriptExecutor) driver)
```

```
                .executeScript(
                        "arguments[0].click();",
                        button);
    }

    public String getLogoutAlertText() {

        Alert alert = WaitUtil.waitForAlert(driver);

        String alertText = alert.getText();

        alert.accept();

        return alertText;
    }

    public boolean isLoginPageDisplayed() {

        return driver.findElement(By.name("uid"))
                .isDisplayed();
    }
}
```

3.3 utilities

ExcelUtil.java

Purpose:

- Read Excel data
- Write test results back to Excel

Methods:

- getSheetData()
- setCellData()
- findRow()
- clearCellData()

```
package utilities;

import java.io.FileInputStream;
import java.io.FileOutputStream;

import org.apache.poi.ss.usermodel.*;
import org.apache.poi.xssf.usermodel.XSSFWorkbook;

public class ExcelUtil {

    private Sheet sheet;
    private Workbook workbook;
    private String filePath;

```

```
    public ExcelUtil(String filePath, String sheetName) {

        try {

            this.filePath = filePath;

            FileInputStream fis = new FileInputStream(filePath);

            workbook = new XSSFWorkbook(fis);

            sheet = workbook.getSheet(sheetName);

            fis.close();

        } catch (Exception e) {

            e.printStackTrace();
        }
    }

    public Object[][] getSheetData() {
        int rows = sheet.getLastRowNum();
        int cols = sheet.getRow(0).getLastCellNum();
        Object[][] data = new Object[rows][cols];

        for (int i = 1; i <= rows; i++) {
            Row row = sheet.getRow(i);

            // Skip null rows (e.g. trailing empty rows in Excel)
            if (row == null) continue;

            for (int j = 0; j < cols; j++) {
                Cell cell = row.getCell(j);

                // Return empty string instead of crashing on null cell
                data[i - 1][j] = (cell == null) ? "" : cell.toString();
            }
        }
        return data;
    }


    //to write data in excel:
    public void setCellData(int rowNum, int colNum, String value) {

        try {

            Row row = sheet.getRow(rowNum);

            if(row == null)
                row = sheet.createRow(rowNum);

            Cell cell = row.getCell(colNum);

            if(cell == null)
```

```
                cell = row.createCell(colNum);

            cell.setCellValue(value);

            FileOutputStream fos = new FileOutputStream(filePath);

            workbook.write(fos);

            fos.close();

        } catch(Exception e) {

            e.printStackTrace();
        }
    }


    public String getCellData(int rowNum, int colNum) {
       try {
              Row row = sheet.getRow(rowNum);

              if (row == null) {
                  return "";
              }

              Cell cell = row.getCell(colNum);

              if (cell == null) {
                  return "";
              }

              return cell.toString();

       }catch(Exception e) {

            e.printStackTrace();
            return "";
        }
    }



    //find row

    public int findRow(String tcId) {

        try {

            int lastRow = sheet.getLastRowNum();

            for (int i = 1; i <= lastRow; i++) {

                Row row = sheet.getRow(i);

                if (row == null) {
```

```
                    continue;
                }

                Cell cell = row.getCell(0); // TC_ID column

                if (cell == null) {
                    continue;
                }

                String cellValue = cell.toString().trim();

                if (cellValue.equalsIgnoreCase(tcId.trim())) {
                    return i;
                }
            }

        } catch (Exception e) {
            e.printStackTrace();
        }

        return -1;
    }


    //clear cell data



    public void clearCellData(int rowNum, int colNum) {

        try {

            Row row = sheet.getRow(rowNum);

            if (row == null) {
                return;
            }

            Cell cell = row.getCell(colNum);

            if (cell == null) {
                return;
            }

            cell.setBlank();

            FileOutputStream fos = new FileOutputStream(filePath);

            workbook.write(fos);

            fos.close();

        } catch (Exception e) {

            e.printStackTrace();
```

```
        }
    }
}
```

WaitUtil.java

Purpose: Provides:

- Explicit Wait
- Fluent Wait
- Alert Wait

```
package utilities;


import java.time.Duration;
import java.util.NoSuchElementException;

import org.openqa.selenium.Alert;
import org.openqa.selenium.By;
import org.openqa.selenium.ElementClickInterceptedException;
import org.openqa.selenium.StaleElementReferenceException;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.FluentWait;
import org.openqa.selenium.support.ui.WebDriverWait;

public class WaitUtil {

    WebDriver driver;
    WebDriverWait wait;

    public WaitUtil(WebDriver driver) {
        this.driver = driver;
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
    }

    public WebElement waitForElementVisible(By locator) {
        return wait.until(
                ExpectedConditions.visibilityOfElementLocated(locator));
    }

    public WebElement waitForElementClickable(By locator) {
        return wait.until(
                ExpectedConditions.elementToBeClickable(locator));
    }

    public static Alert waitForAlert(WebDriver driver) {

        WebDriverWait wait =
                new WebDriverWait(driver, Duration.ofSeconds(10));

        return wait.until(
```

```
                ExpectedConditions.alertIsPresent());
    }


    //fluent visibility wait

    public WebElement waitForElementVisibleFluent(By locator) {

        FluentWait<WebDriver> wait =
                new FluentWait<>(driver)
                        .withTimeout(Duration.ofSeconds(20))
                        .pollingEvery(Duration.ofSeconds(2))
                        .ignoring(NoSuchElementException.class);

        return wait.until(driver ->
                driver.findElement(locator).isDisplayed()
                        ? driver.findElement(locator)
                        : null);
    }



    public WebElement waitForElementClickableFluent(By locator) {

        FluentWait<WebDriver> wait =
                new FluentWait<>(driver)
                        .withTimeout(Duration.ofSeconds(20))
                        .pollingEvery(Duration.ofSeconds(2))
                        .ignoring(NoSuchElementException.class)
                        .ignoring(StaleElementReferenceException.class)
                        .ignoring(ElementClickInterceptedException.class);

        return wait.until(driver -> {

            WebElement element = driver.findElement(locator);

            return (element.isDisplayed() && element.isEnabled())
                    ? element
                    : null;
        });
    }

}
```

ExtentManager.java

Purpose:

Configures Extent Reports.

```
package utilities;
```

```
import com.aventstack.extentreports.ExtentReports;
import com.aventstack.extentreports.reporter.ExtentSparkReporter;

public class ExtentManager {

    private static ExtentReports extent;

    public static ExtentReports getInstance() {

        if(extent == null) {

            String reportPath =
                    System.getProperty("user.dir")
                    + "/reports/ExtentReport.html";

            ExtentSparkReporter spark =
                    new ExtentSparkReporter(reportPath);

            spark.config().setReportName(
                    "Guru99 Bank Automation Report");

            spark.config().setDocumentTitle(
                    "Automation Execution Report");

            extent = new ExtentReports();

            extent.attachReporter(spark);

            extent.setSystemInfo(
                    "Project",
                    "Guru99 Bank");

            extent.setSystemInfo(
                    "Tester",
                    "Shahid");
        }

        return extent;
    }
}
```

ScreenshotUtil.java

Purpose:

Captures screenshots on failure.

```
package utilities;

import java.io.File;
import java.io.IOException;

import org.apache.commons.io.FileUtils;
import org.openqa.selenium.*;
```

```
public class ScreenshotUtil {

    public static String captureScreenshot(
            WebDriver driver,
            String testName) {

        String path =
                "screenshots/"
                + testName
                + "_"
                + System.currentTimeMillis()
                + ".png";

        File src =
                ((TakesScreenshot) driver)
                .getScreenshotAs(OutputType.FILE);

        try {

            FileUtils.copyFile(src,
                    new File(path));

        } catch (IOException e) {

            e.printStackTrace();
        }

        return path;
    }
}
```

LoggerUtil.java

Purpose:

Handles logging operations.

```
package utilities;

import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class LoggerUtil {

    public static Logger logger =
            LogManager.getLogger();
}
```

ConfigReader.java

Purpose:

Reads configuration properties.

```
package utilities;

import java.io.FileInputStream;
import java.io.IOException;
import java.util.Properties;

public class ConfigReader {

    private static Properties prop;

    static {

        try {

            FileInputStream fis = new FileInputStream(
                    "src/test/resources/config/config.properties");

            prop = new Properties();

            prop.load(fis);

        } catch (IOException e) {

            e.printStackTrace();
        }
    }

    public static String getProperty(String key) {

        return prop.getProperty(key);
    }
}
```

4. Framework Architecture

The architecture diagram below represents the layered structure of the framework, from test execution down to the application under test.

4.1 Architecture Flow

```
TestNG Tests
      |
      v
Page Objects
      |
      v
Utilities
      |
      v
Selenium WebDriver
      |
      v
Guru99 Application
```

5. Design Patterns Used

5.1 Page Object Model (POM)

Separates test scripts from page-specific logic.

5.2 Data-Driven Testing

Uses Excel files for test data.

5.3 Singleton Driver Pattern

Implemented in DriverFactory.

6. Excel Data Management

Test data for the framework is stored in Excel workbooks. Apache POI is used to read input data and write back test execution results, allowing test results to be updated dynamically.

- Test data stored in Excel
- Apache POI used for reading/writing
- Test results updated dynamically

6.1 Sample Data Files

CustomerData.xlsx

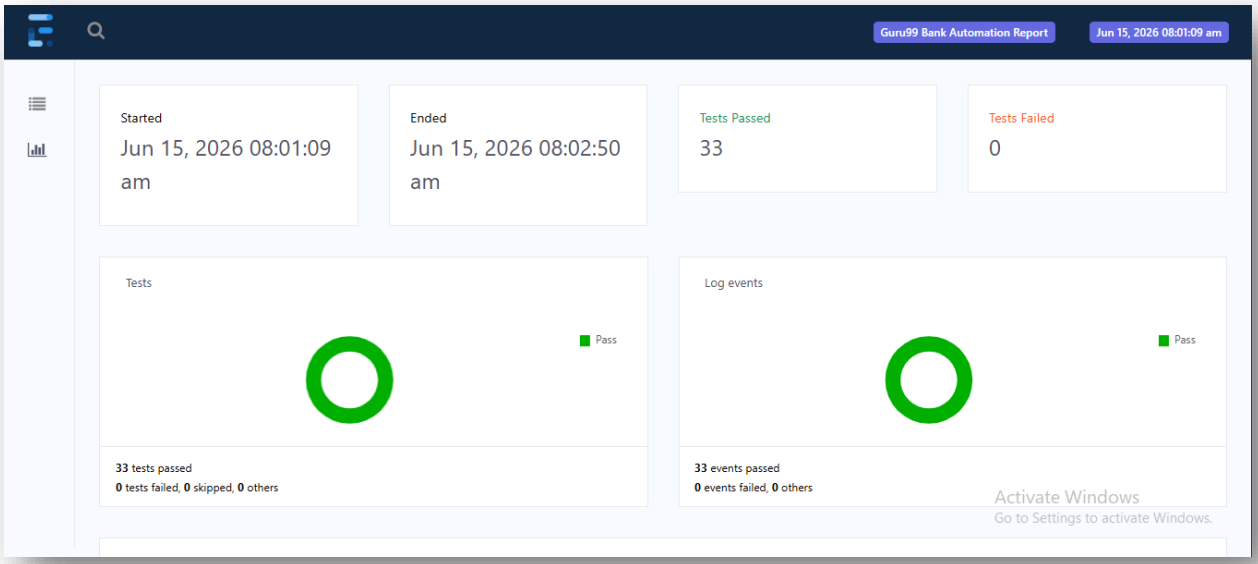
A	B	C	D	E	F	G	H	I	J	K	L	M	N
TestCaseI	Name	DOB	Address	City	State	PIN	Phone	Email	Password	Customer	Execution	expectedresult	
TC001	Shahid	01-01-199	Jamshedp	Jamshedp	Jharkhanc	831001	987654321	shahid1@	test123			valid	
TC002	Rakesh	02-02-199	Ranchi	Ranchi	Jharkhanc	834001	999999999	rakesh@g	test123			valid	
TC003	Pooja	16-11-199	Pune	Pune	Maharashtra		991234567	pooja@gn	test123			invalid	
TC004	Deepak	19-02-199	Coimbato	Coimbato	Tamil Nad	600001	9945678123		test123			invalid	

DepositData.xlsx

A	B	C	D
AccountID	Amount	description	status
184459	24000	testing deposit	valid
184028a	6000	testing deposit	invalid
184459	gfdg66	testing deposit	invalid
	6000	testing deposit	invalid

7. Reporting

Extent Reports are generated after each test execution, providing a clear summary of the test run.



7.1 Report Contents

- Pass/Fail status
- Screenshots on failure
- Execution time

8. Docker Integration

[Insert Screenshot Here]

Insert Docker screenshot

8.1 Example Commands

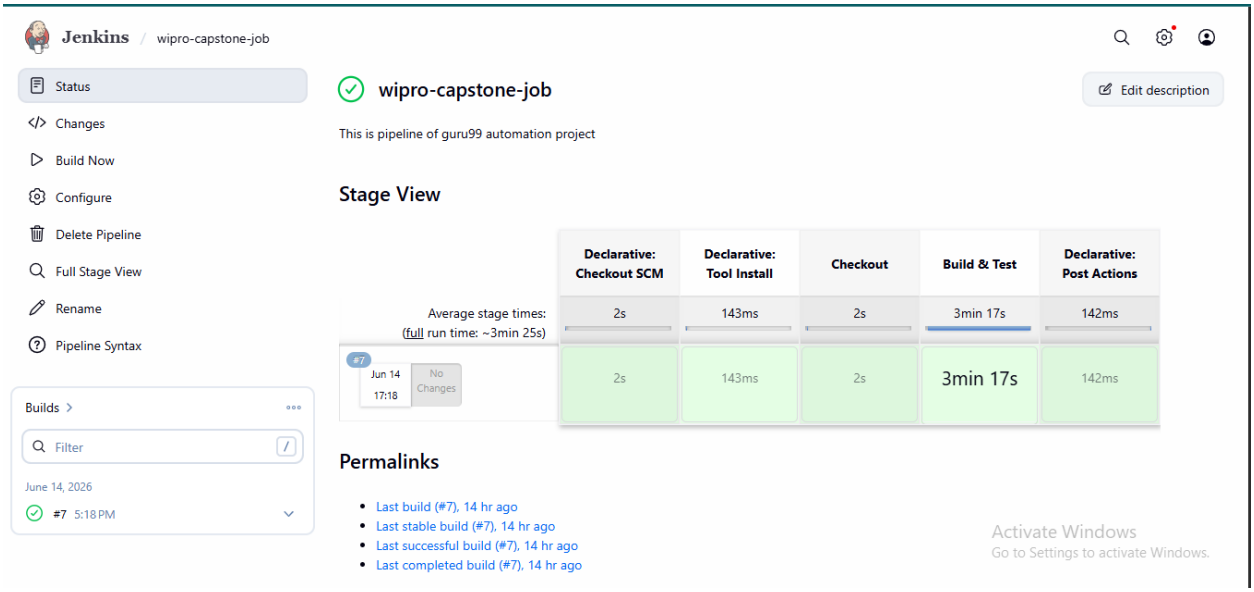
```
docker build -t guru99-automation .
docker run --rm guru99-automation
```

8.2 Dockerfile Explanation

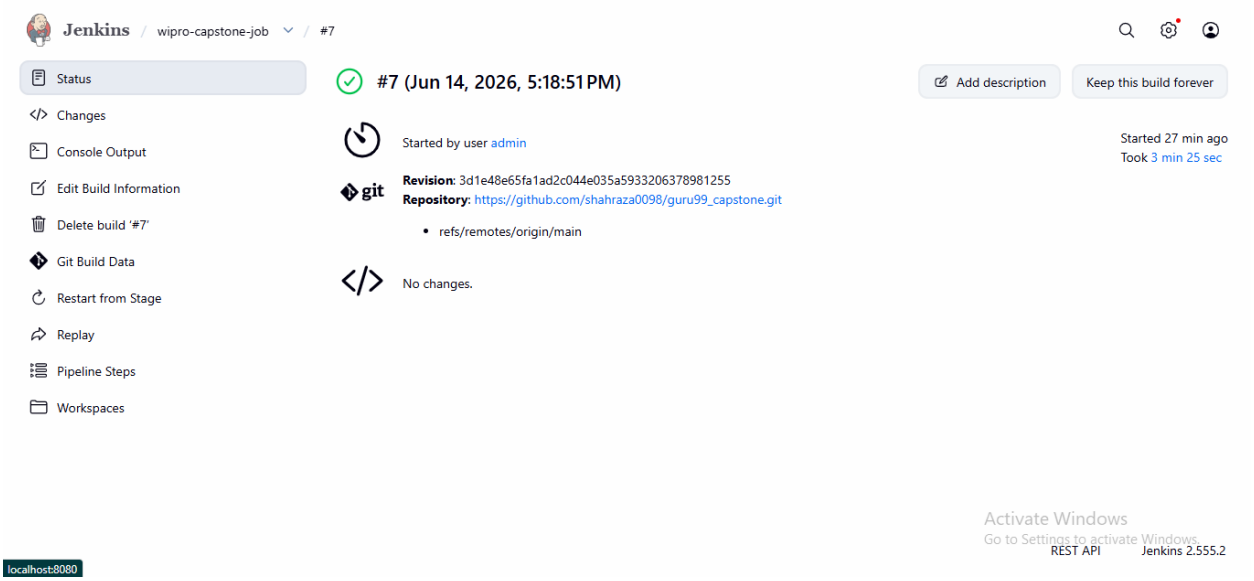
```
FROM maven:3.9.6-eclipse-temurin-21

# Install dependencies
RUN apt-get update && apt-get install -y \
    wget \
    gnupg \
    unzip \
    curl \
    && rm -rf /var/lib/apt/lists/*

# Install Chrome (modern way — no deprecated apt-key)
RUN wget -q -O /tmp/google-chrome.deb \
    https://dl.google.com/linux/direct/google-chrome-stable_current_amd64.deb \
    && apt-get update \
    && apt-get install -y /tmp/google-chrome.deb \
    && rm /tmp/google-chrome.deb \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY . .

RUN mvn clean install -DskipTests

CMD ["mvn", "test", "-Dbrowser=chrome"]
```

9. Jenkins Integration

9.1 Jenkins Dashboard

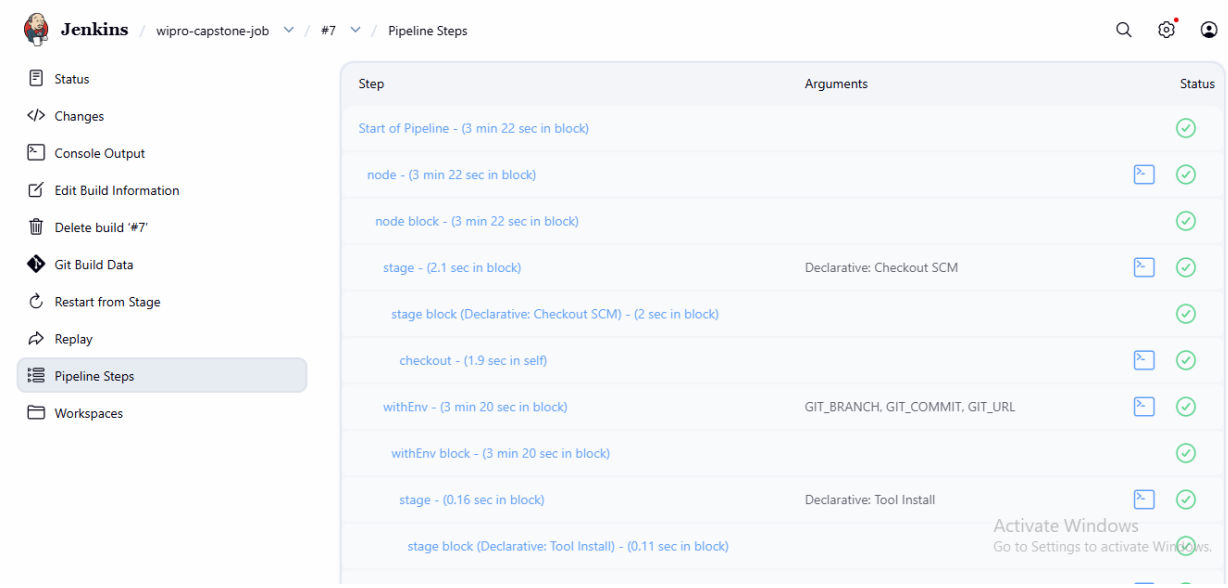


9.2 Build Success



9.3 Console Output

9.4 Pipeline



9.5 Example Pipeline

```
pipeline {
    agent any

    tools {
        maven 'Maven-3.9'
    }

    stages {

        stage('Checkout') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/shahraza0098/guru99_capstone.git'
            }
        }

        stage('Build & Test') {
            steps {
                dir('guru99_automation') {
                    bat 'mvn clean test'
                }
            }
        }
    }

    post {
        success {
            echo 'Build Successful'
        }

        failure {
            echo 'Build Failed'
        }
```

```
    }
}
```

10. Defect Report

Defect ID	Module	Description	Severity	Status
DEF-001	Deposit	HTTP ERROR 500 after deposit	High	Open
DEF-002	Withdrawal	Negative amount accepted	High	Open
DEF-003	Deposit	Negative amount accepted	High	Open
DEF-004	Edit Customer	User redirect to black white page when edit customer records	Medium	Open
DEF-005	Fund Transfer	Fund Transfer feature accepting negative amount	High	Open

11. Challenges Faced

- Dynamic customer IDs
- Alert handling
- Scroll and click interception
- HTTP 500 errors
- Data-driven execution
- Docker/Jenkins integration

12. Conclusion

The framework successfully automates the Guru99 Banking application using Selenium WebDriver and TestNG while supporting CI/CD through Jenkins and Docker.